TP1 Pointeur

Exercice 1 :

```
#include <stdio.h>
#include <stdlib.h>


int main() {
   int *ptr;


   // #1 Allocation de mémoire dynamique pour le pointeur ptr
   ptr = (int *)malloc(sizeof(int));


   // #2 Vérification si l'allocation de mémoire a réussi
   if (ptr == NULL) {
      printf("Erreur d'allocation de mémoire\n");
      return 1;    }
   *ptr = 5;


   // Affichage de la valeur pointée par ptr
   printf("La valeur pointée par ptr est : %d\n", *ptr);


   // Libération de la mémoire allouée
   free(ptr);


   return 0;
}
```

Exercice 2:

1. Le code est correct, il calcule la somme des valeurs pointées par les pointeurs **ptr1** et **ptr2**, qui sont les adresses des variables **x** et **y** respectivement. Le résultat est ensuite affiché.

2. ***Différence entre *ptr et ptr :***

- **ptr** est un pointeur qui contient une adresse mémoire.
- ***ptr** est la valeur stockée à l'adresse mémoire pointée par **ptr**.

3.Correction du code: Le code présente une erreur car il tente de déréférencer un pointeur NULL, ce qui entraîne un comportement indéfini. Pour corriger cela, nous devons soit allouer de la mémoire pour **ptr** avant de le déréférencer, soit vérifier si **ptr** n'est pas NULL avant de le déréférencer.

Voici une correction en vérifiant si **ptr** n'est pas NULL avant de le déréférencer :

```
#include <stdio.h>


int main() {
   int *ptr = NULL;


   if (ptr != NULL) {
      printf("La valeur pointée par ptr est : %d\n", *ptr);
   } else {
      printf("Le pointeur ptr est NULL\n");
   }
   return 0;
}
```

4.Qu'est-ce qu'un pointeur NULL ? À quoi sert-il?

Un pointeur NULL est un pointeur qui ne pointe sur aucune adresse mémoire valide. Il est souvent utilisé pour indiquer l'absence d'une adresse valide, par exemple lorsqu'une fonction de recherche échoue et ne trouve pas un élément spécifié. Utiliser un pointeur NULL peut aider à éviter des comportements indéfinis ou des plantages du programme.

5.Explication de l'opérateur & en C : L'opérateur **&** en C est l'opérateur d'adresse. Lorsqu'il est utilisé avec une variable, il retourne l'adresse mémoire de cette variable. Par exemple, si **x** est une variable, alors **&x** donne l'adresse mémoire de **x**.

Exercice3:

```
#include <stdio.h>

int fPlusGrand(int *tableau, int taille) {
   int max = *tableau; // Initialisation du maximum avec le premier élément du tableau
   int *ptr = tableau; // Initialisation du pointeur au début du tableau

   // Parcours du tableau pour trouver le plus grand élément
   for (int i = 1; i < taille; i++) {
      if (*(ptr + i) > max) {
         max = *(ptr + i);
      }
   }
   return max;
}

int main() {

   int tableau[] = {5, 100, 7, 8, 25,100,2};
   int taille = sizeof(tableau) / sizeof(tableau[0]);
   // Appel de la fonction pour trouver le plus grand élément
   int plusGrand = fPlusGrand(tableau, taille);
   printf ("Le plus grand élément du tableau est : %d\n", plusGrand);

   return 0;
}
```